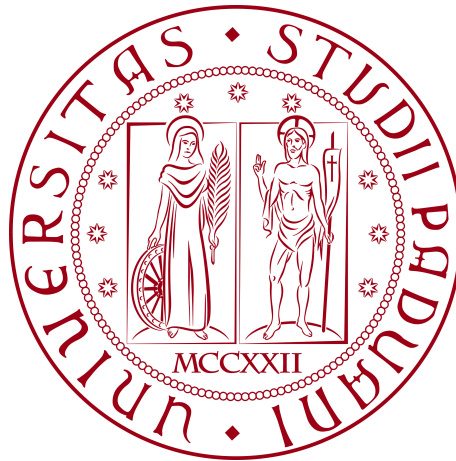


University of Padua

DEPARTMENT OF MATHEMATICS “TULLIO LEVI-CIVITA”

MASTER DEGREE IN COMPUTER SCIENCE



When Does Structural Reasoning Help? Structural
Generalization in Neurosymbolic Learning

Master's Thesis

Supervisor

Prof. Luciano Serafini

Co-Supervisor

Prof. Roberto Confalonieri

Candidate

Marco Brugin

ID Number: 2122864

ACADEMIC YEAR 2026-2027

“We can only see a short distance ahead, but we can see plenty there that needs to be done.” — Alan Turing

Acknowledgements

First and foremost, I would like to express my sincere gratitude to Prof. Serafini and Prof. Confalonieri for their guidance and support throughout the development of my Master’s thesis project. Their knowledge, advice, and availability have been fundamental to my academic growth and to the development of this thesis. I am particularly grateful for the opportunity to work in the field of Artificial Intelligence, a rapidly evolving area that has allowed me to deepen my technical knowledge while developing a greater understanding of the opportunities and challenges that arise from the development of intelligent systems.

I would also like to thank my entire family for their constant support throughout these years. Their presence, encouragement, and love have accompanied me throughout this journey, especially during the more difficult moments. Completing this degree required commitment, perseverance, and many sacrifices, and I am deeply grateful to my family for always being there for me and for supporting me in every step of the way.

A special thank you goes to my girlfriend Laura, who has been by my side throughout this entire journey. Her constant support, patience, and love have helped me through both the most challenging moments and the achievements that made this journey worthwhile. I am deeply grateful for everything she has shared with me and for always being there when I needed her.

This thesis marks the conclusion of an important chapter of my academic journey and, at the same time, the culmination of a path that has allowed me to explore Artificial Intelligence in greater depth. During these years, I have had the opportunity to learn not only about algorithms, models, and technologies, but also about the broader questions surrounding the development and use of intelligent systems. The rapid evolution of Artificial Intelligence

makes it difficult to predict where this field will lead in the future, but it also makes clear that there is still much to discover, understand, and build.

As I reach the end of this Master's degree, I look back with gratitude at the people who supported me and at the experiences that shaped this journey. I hope to carry forward the knowledge, curiosity, and passion developed during these years, and to continue exploring the possibilities of a field that is still only beginning to reveal its full potential.

Padova, 2027

Marco Brugin

Abstract

This thesis investigates the use of Structured Logic Tensor Networks (LTNs) for neuro-symbolic learning. The goal is to combine neural learning with explicit background logical knowledge. The proposed approach addresses an AI classification task with limited and heterogeneous training data. Entities and concepts are represented through learnable continuous embeddings. Logical relations and domain constraints are expressed using first-order logic. These logical formulas are converted into differentiable constraints using fuzzy logic semantics. The model jointly learns from labeled examples and the available background knowledge. A structured representation is used to organize concepts, relations, and logical dependencies. This allows domain knowledge to guide the learning process rather than relying only on observed data. The approach is evaluated against conventional neural network baselines. Experiments investigate both predictive accuracy and logical consistency of the learned model. Additional experiments analyze the effect of increasing or reducing the amount of background knowledge. The results examine whether logical constraints improve generalization, particularly with limited training data. The thesis also discusses computational and modeling challenges associated with differentiable logical reasoning. Overall, the study demonstrates the potential of Structured LTNs for integrating learning and reasoning in AI systems.

Table of contents

List of listings	xi
1 Introduction	1
1.1 Logic Tensor Networks	2
1.2 From Flat Domains to Structured Data	3
1.3 Structural Logic Tensor Networks	5
1.4 Syntax, Semantics, and Tensorial Interpretation	6
1.5 Motivation and Research Gap	7
1.6 Research Questions	8
1.7 Contributions of the Thesis	9
1.8 Thesis Organization	10
2 Theoretical Background	12
2.1 Neurosymbolic Artificial Intelligence	12
2.2 First-Order Logic	13
2.3 Fuzzy Logic and Differentiable Semantics	14
2.4 Logic Tensor Networks	15
2.5 Structural Information in Learning	16
2.6 Structural Logic Tensor Networks	17
2.7 Systematic Generalization	18
2.8 Structural Dependency	19
2.9 Structural Generalization and Data Availability	20
2.10 Research Gap	20

TABLE OF CONTENTS

3	Structural Logic Tensor Networks	22
3.1	Motivation and Limitations of Standard LTN for Structured Data	23
3.2	Structural Dimensions	24
3.3	Structural Variables	24
3.4	Structural Relations	25
3.5	Syntax and Semantics	26
3.6	Tensor Representation and Named Axes	27
3.7	Logical Reasoning over Structural Information	27
3.8	Temporal Structures as a Case Study	28
3.9	Relationship between LTN and sLTN	29
3.10	Limitations and Open Issues of sLTN	30
4	Experimental Methodology	31
4.1	Experimental Objective and Research Questions	31
4.2	Experimental Task	32
4.3	Structural Dependency	33
4.3.1	Low Structural Dependency	33
4.3.2	Medium Structural Dependency	34
4.3.3	High Structural Dependency	35
4.4	Dataset Construction	36
4.5	Structural Signatures	37
4.6	Structural Train–Test Split	38
4.7	Example Disjointness and Leakage Control	38
4.8	Model Configurations	39
4.8.1	Neural Baseline	39
4.8.2	Logic Tensor Network	40
4.8.3	Structural Logic Tensor Network	40
4.9	Logical Constraints	41
4.10	Training Data Availability	42
4.11	Experimental Matrix	42

TABLE OF CONTENTS

4.12	Random Seeds and Reproducibility	43
4.13	Evaluation Metrics	44
4.13.1	Seen Accuracy	44
4.13.2	Unseen Accuracy	44
4.13.3	Generalization Gap	44
4.14	Data Efficiency Analysis	45
4.15	Statistical Analysis	45
4.16	Experimental Hypotheses	46
4.16.1	H1 — Structural Generalization	46
4.16.2	H2 — Dependence on Structural Dependency	46
4.16.3	H3 — Training Data Availability	46
4.16.4	H4 — Structural Relevance	47
4.17	Implementation and Experimental Reproducibility	47
4.18	Experimental Controls	48
4.19	Experimental Interpretation Protocol	48
5	Experimental Results	50
6	Analysis of Structural Generalization	51
7	Discussion	52
8	Conclusions and future research	53
	Bibliography	54

List of figures

List of tables

4.1	Dataset sizes used in the experiments.	37
4.2	Experimental conditions.	43

List of listings

LIST OF LISTINGS

Chapter 1

Introduction

Artificial intelligence systems are increasingly required not only to learn from data, but also to reason about the knowledge acquired from their environment. Modern machine learning methods have demonstrated remarkable capabilities in processing complex and high-dimensional data, including images, natural language, and other forms of unstructured information. However, learning directly from data does not by itself provide an explicit representation of knowledge or a principled mechanism for expressing logical constraints. Conversely, symbolic artificial intelligence provides powerful formalisms for representing and manipulating structured knowledge, but traditionally relies on explicitly defined symbols and rules and therefore faces difficulties when dealing with noisy, incomplete, or high-dimensional data. The integration of learning and reasoning has consequently become a central topic in neurosymbolic artificial intelligence [1]. Within this context, Logic Tensor Networks (LTNs) provide a framework for integrating first-order logic with differentiable neural computation. LTN introduces a many-valued, end-to-end differentiable logical language, called Real Logic, in which the elements of a first-order logic signature are grounded onto data through neural computational graphs. Logical formulas are assigned truth values in the interval $[0, 1]$, allowing logical constraints to participate directly in gradient-based learning [1]. This provides a common framework in which learning from data and reasoning with background knowledge can be expressed within the same formalism. More recently, the Structural Logic Tensor Networks (sLTN) framework extends the original LTN formulation to explicitly represent structural organization in data. While conventional LTN is primarily formulated for domains consisting of collections of individuals, many real-world datasets possess additional dimensions such as temporal order, sequence position, or graph structure. sLTN introduces

structural dimensions, structural variables, and structural relations as explicit components of the logical language, allowing such organization to be represented and reasoned about at the logical level [5]. This thesis investigates this extension from a research perspective, focusing not only on how structural information can be represented, but also on whether explicitly modelling such information can improve the generalization of neurosymbolic learning systems.

1.1 Logic Tensor Networks

Logic Tensor Networks were introduced as a neurosymbolic framework designed to combine the representational capabilities of first-order logic with the learning capabilities of neural networks. The central idea is to interpret the symbols of a logical language using differentiable tensorial representations, so that logical knowledge can be incorporated directly into a machine learning system [1]. Traditional machine learning approaches generally represent knowledge implicitly through numerical parameters learned from data. Such approaches are particularly effective when large amounts of labelled or unlabelled data are available, but the resulting representations do not necessarily expose the logical relationships that hold between concepts. Symbolic approaches, on the other hand, represent knowledge explicitly using concepts, relations, and logical rules. This makes symbolic representations suitable for reasoning and knowledge manipulation, but classical symbolic systems are generally less suited to directly processing raw, noisy, or high-dimensional observations [1]. Neurosymbolic approaches attempt to bridge this distinction by combining symbolic representations with sub-symbolic learning. In this setting, symbolic knowledge can provide constraints and relationships that guide learning, while neural models provide mechanisms for learning representations from data. A fundamental issue in such systems is therefore the grounding of symbolic entities into concrete data representations. LTN addresses this problem by assigning differentiable tensor representations and functions to the elements of a first-order logical language [1]. The LTN framework is based on Real Logic, a many-valued and differentiable extension of first-order logic. In this setting, logical formulas are not restricted to the classical truth values of true and false. Instead, their truth values are represented numerically

in the interval $[0, 1]$. Logical connectives and quantifiers are implemented using differentiable fuzzy-logic operations, allowing the satisfaction of logical formulas to be optimized using gradient-based methods [1]. This formulation leads to an important reinterpretation of learning. Rather than defining learning solely as the minimization of a task-specific numerical loss, LTN allows the learning problem to be expressed through a collection of logical formulas whose degree of satisfaction is optimized. The parameters of the underlying neural computational graphs can consequently be learned so as to maximize the overall truth value of the specified knowledge [1]. The resulting framework can express a variety of learning and reasoning tasks. The original LTN work demonstrates applications including data clustering, multi-label classification, relational learning, query answering, semi-supervised learning, regression, and embedding learning [1]. The common feature of these tasks is that data-driven learning and logical constraints can be expressed within a single differentiable framework. An LTN model can therefore be viewed as consisting of two interacting components. On the one hand, neural computation provides continuous representations and differentiable mappings from data to these representations. On the other hand, the logical layer specifies relationships and constraints between the represented entities. The grounding connects these two levels by mapping logical terms and predicates to differentiable tensor operations [1]. This combination is particularly relevant when background knowledge is available. Instead of treating the available knowledge as an external post-processing mechanism, LTN incorporates it directly into the learning objective. The logical structure can consequently influence the learned representations while the neural components provide the flexibility required to learn from numerical data [1].

1.2 From Flat Domains to Structured Data

Despite the generality of the LTN framework, its original formulation is primarily oriented towards domains that can be described as collections of individuals and relations between them. This setting is appropriate for many relational learning problems, where entities can be represented independently and logical predicates describe their properties or relationships. However, a large class of modern datasets contains an additional structural organization that

cannot be naturally described only through relations between independent individuals [5]. Examples include temporal sequences, where observations are associated with ordered time steps; sequential data, where the position of an element within a sequence is significant; and graphs, where the connectivity between nodes provides essential information about the domain. In these settings, the position or structural context of an observation is itself meaningful. Treating such information merely as an additional property of an individual does not necessarily provide a natural representation of the underlying structure [5]. Consider a temporal dataset represented as a tensor. One axis may correspond to different individuals or samples, while another axis represents time. The temporal axis has a meaning that is different from the identity of the individuals contained in the tensor. Similarly, in a sequence, the position of an element is not simply another individual in the domain: it represents its location within an ordered structure. In a graph, the organization of nodes and their connectivity introduces another form of structure that is not captured solely by treating nodes as an unordered collection [5]. This distinction motivates the introduction of explicit structural information into the logical language. The sLTN framework addresses this problem by extending LTN with structural dimensions that represent named dimensions of structured objects. Examples of structural dimensions include time steps, sequence positions, and graph nodes [5]. Structural dimensions are therefore conceptually different from ordinary logical sorts and individuals. An ordinary first-order variable ranges over individuals in a domain, whereas a structural variable ranges over positions associated with a structural dimension. This distinction allows logical formulas to refer explicitly to the organization of data rather than encoding structural information indirectly [5]. The introduction of structural dimensions also makes it possible to define structural relations. A structural relation can describe relationships between positions within a structure. For example, a relation such as

$$\text{next}(t, t')$$

can express that t' is the position immediately following t in a temporal or sequential dimension. Such relations can then be used inside logical formulas to express constraints involving

neighbouring or otherwise structurally related positions [5]. This extension is important because structural relationships can influence the interpretation of a formula. A constraint may not simply state that a property holds for an individual, but that it holds at a particular structural position, or that properties at two positions are related according to the structure of the underlying data. sLTN consequently provides a way to express temporal, sequential, and relational constraints directly within the logical language [5].

1.3 Structural Logic Tensor Networks

Structural Logic Tensor Networks extend the LTN framework by making structural dimensions first-class elements of the logical language. The framework introduces an explicit distinction between the logical domain of individuals and the structural organization associated with tensors. This allows logical expressions to refer to positions within structured data while retaining the differentiable fuzzy semantics of LTN [5]. The formal language of sLTN includes ordinary first-order elements such as sorts, constants, variables, functions, and predicates, together with structural elements such as structural variables and structural relations. Structural dimensions provide the link between the logical description and the axes of the underlying tensors [5]. A structural dimension can be understood as a named axis associated with a particular type of organization in the data. For example, a dimension called *time* may identify the temporal axis of a tensor, while another dimension may represent positions in a sequence. The structural dimension is not merely metadata: it participates in the interpretation of logical expressions by determining how tensorial data are aligned, selected, masked, or aggregated [5]. Structural variables are used to quantify over the positions of these dimensions. In contrast to ordinary first-order variables, which range over individuals, structural variables range over indices associated with structural dimensions. This distinction allows formulas to make explicit statements about the location of data within a structured object [5]. Structural relations provide a further level of expressiveness. They allow relationships between structural positions to be represented explicitly. For example, a temporal relation can identify consecutive time steps, while other structural relations can describe more general dependencies between positions. These relations can be Boolean or

fuzzy and can therefore be integrated into the differentiable semantics of the framework [5]. An important consequence of this design is that structural information becomes part of the logical theory rather than being handled exclusively by the underlying neural architecture. The logical language can therefore express constraints whose meaning depends directly on the organization of the data. This is particularly relevant for problems in which temporal or sequential dependencies are part of the knowledge that should guide learning [5]. The sLTN framework retains the central principle of LTN: logical expressions are interpreted through differentiable tensor operations. Thus, introducing structural dimensions does not replace the original tensorial semantics but extends it to account for additional axes of organization. The resulting framework can align logical expressions with tensor dimensions and use structural information when performing selection, masking, or aggregation operations [5]. An important theoretical property of sLTN is that the original LTN framework can be recovered as a special case. When no structural dimensions are introduced, the structural extension does not add additional structure to the interpretation, and the framework reduces to the original LTN setting [5].

1.4 Syntax, Semantics, and Tensorial Interpretation

A central aspect of sLTN is the explicit separation between the syntactic and semantic levels of the framework. The language first defines the symbols and expressions that are permitted by the theory. These expressions are then parsed and validated before being interpreted over tensorial data [5]. The signature specifies the vocabulary available to a theory. It contains the sorts, dimensions, constants, variables, functions, predicates, and structural relations required to construct formulas. This explicit signature provides a formal description of the entities and relationships that can appear in a logical theory [5]. The syntactic layer is followed by formula parsing and validation. This separation is important because it distinguishes the logical structure of an expression from the numerical representation used to evaluate it. A formula can therefore be treated as a symbolic object before being mapped to tensor operations by the interpretation mechanism [5]. At the semantic level, the symbols of the logical language are assigned tensorial interpretations. Constants and variables are

grounded into tensor representations, while predicates and functions are interpreted through differentiable operations. Logical connectives and quantifiers are then evaluated using fuzzy semantics, producing numerical truth values [1, 5]. Structural dimensions introduce an additional component into this interpretation. A structural dimension determines an axis of the tensorial representation, while structural variables identify positions along that axis. Structural relations determine relationships between such positions. The interpretation can consequently use the structural information to align tensors and to perform operations that respect the organization encoded by the data [5]. This tensorial interpretation is essential for maintaining differentiability. Logical reasoning is not implemented as a separate symbolic process disconnected from learning. Instead, formulas are translated into differentiable numerical operations whose outputs can contribute to an optimization objective. As in LTN, the truth value of a formula can therefore influence the learning of the parameters involved in its grounding [1, 5]. The separation between syntax and semantics also provides a modular view of the framework. The logical theory specifies what can be expressed, while the interpretation specifies how those expressions are connected to the underlying data. This distinction becomes particularly useful for structured data because the same logical concepts can be associated with different tensorial organizations through their structural dimensions [5].

1.5 Motivation and Research Gap

The introduction of structural dimensions and relations provides a mechanism for representing information that is intrinsic to the organization of structured data. However, the availability of such a representation does not by itself establish whether explicit structural reasoning provides a measurable advantage during learning. In particular, it remains important to determine the conditions under which structural information can improve the ability of a neurosymbolic model to generalize learned knowledge to structural configurations that were not observed during training. This question is particularly relevant because a model may achieve good predictive performance without explicitly representing structure. A purely neural model may learn statistical regularities from the observations, while a conventional

LTN may incorporate logical constraints without explicitly modelling structural positions and relations. Consequently, any improvement observed with sLTN should not automatically be attributed to the presence of explicit structural reasoning. This thesis therefore focuses on the relationship between structural information and generalization. Rather than assuming that explicit structural reasoning is beneficial in every setting, the study investigates whether its contribution depends on the extent to which the target task relies on structural relationships. In addition, the study examines whether this relationship changes when the amount of available training data is reduced.

The central research problem can therefore be formulated as follows:

When does explicit structural reasoning improve the generalization of neurosymbolic models to previously unseen structural configurations?

The experimental analysis is designed to identify conditions under which explicit structural representations provide a measurable benefit, as well as conditions in which their contribution is limited. This allows the study to characterize the role of structural reasoning rather than simply comparing the overall predictive performance of different models.

1.6 Research Questions

The main objective of this thesis is to investigate the conditions under which explicitly representing structural information can improve the ability of neurosymbolic models to generalize beyond the structural configurations observed during training. To address this objective, the thesis considers the following research questions:

- **RQ1 — Structural Generalization.**

To what extent does explicit structural reasoning affect the generalization of neurosymbolic models to previously unseen structural configurations?

- **RQ2 — Model Comparison.**

How does the performance of a Structural Logic Tensor Network compare with that of a conventional Logic Tensor Network and a neural baseline when generalizing to previously unseen structural configurations?

- **RQ3 — Conditions for Structural Benefit.**

How does the effect of explicit structural reasoning vary with the degree of structural dependency of the task and the amount of available training data?

These research questions are addressed through a controlled experimental evaluation involving a neural baseline, a conventional Logic Tensor Network, and a Structural Logic Tensor Network incorporating explicit structural dimensions and relations.

1.7 Contributions of the Thesis

The primary contribution of this thesis is a systematic experimental investigation of the conditions under which explicit structural representations contribute to structural generalization in neurosymbolic learning. The experimental framework provides a controlled comparison between three model settings: a neural baseline, a conventional Logic Tensor Network, and a Structural Logic Tensor Network incorporating explicit structural dimensions and relations. This comparison provides a basis for separating the contribution of logical constraints from that of explicit structural representations. A further contribution is the design of controlled structural generalization scenarios in which the structural configurations encountered during evaluation differ from those observed during training. This experimental setting provides a direct means of assessing whether models can generalize learned knowledge across changes in structural organization. The thesis also investigates how the contribution of explicit structural reasoning changes with the structural dependency of the task. By considering tasks with different degrees of dependence on structural relationships, the study aims to characterize the conditions under which explicit structural representations become useful and the conditions under which their contribution is limited. Finally, the experimental evaluation considers different amounts of available training data. This analysis investigates whether the effect of explicit structural reasoning changes as supervision becomes more limited, with

particular attention to tasks in which structural relationships are important for the target prediction.

1.8 Thesis Organization

The remainder of this thesis is organized as follows:

- **Chapter 2** introduces the theoretical background required for the study, including neurosymbolic learning, first-order logic, fuzzy logic, Logic Tensor Networks, and the principles underlying structured data representations.
- **Chapter 3** presents the Structural Logic Tensor Network framework in greater detail, focusing on its syntax, semantics, structural dimensions, structural variables, structural relations, and tensorial interpretation.
- **Chapter 4** describes the research methodology adopted in this thesis. It introduces the experimental task, the considered models, the structural configurations used during training and testing, the different levels of structural dependency, the amounts of training data considered, the evaluation metrics, and the experimental protocols used to address the research questions.
- **Chapter 5** presents the experimental evaluation of structural generalization. The chapter investigates the performance of the considered models on structural configurations observed during training as well as on previously unseen configurations. It also examines model performance under different levels of structural dependency and different amounts of training data.
- **Chapter 6** analyses the experimental results, with particular emphasis on the relationship between structural dependency, model performance, structural generalization, and training data availability.
- **Chapter 7** discusses the findings in relation to the research questions and hypotheses. Particular attention is given to identifying the conditions under which explicit struc-

tural reasoning provides measurable benefits, as well as the conditions under which its contribution is limited.

- **Chapter 8** summarizes the main findings of the thesis, discusses its limitations, and outlines directions for future research, including the potential extension of the experimental analysis to additional structured domains and neurosymbolic learning settings.

Chapter 2

Theoretical Background

This chapter introduces the theoretical concepts underlying the research presented in this thesis. It first discusses neurosymbolic artificial intelligence and the complementary roles of neural and symbolic approaches. The logical foundations required for Logic Tensor Networks are then introduced, followed by fuzzy and differentiable semantics. The chapter subsequently presents Logic Tensor Networks (LTN) and Structural Logic Tensor Networks (sLTN), with particular attention to the representation of structural information. Finally, the concepts of systematic generalization and structural dependency are introduced to motivate the research question investigated in this thesis.

2.1 Neurosymbolic Artificial Intelligence

Artificial intelligence has developed through different approaches to learning and knowledge representation. Sub-symbolic methods, particularly neural networks, learn numerical representations from data and have demonstrated strong capabilities in tasks involving complex and high-dimensional observations. Symbolic approaches, in contrast, represent knowledge explicitly through symbols, logical rules, and structured relationships. These approaches provide different advantages. Neural models can learn representations directly from examples, but their ability to generalize to configurations that differ systematically from those observed during training is not guaranteed [4]. Symbolic systems provide explicit representations of knowledge and relationships, but traditional symbolic methods are not naturally designed to learn directly from noisy or high-dimensional data.

Neurosymbolic artificial intelligence aims to combine these two paradigms by integrating neu-

ral learning with symbolic knowledge and reasoning [2]. The general objective is to exploit the representational and learning capabilities of neural models while retaining the ability to express structured knowledge explicitly. Logic Tensor Networks follow this approach by combining first-order logical representations with differentiable tensor computation. Logical symbols are grounded into numerical representations and differentiable functions, allowing logical knowledge to participate directly in learning [1].

This perspective is particularly relevant when the information required to solve a task includes relationships between observations. In such cases, the question is not only whether a model can learn from the observed data, but also whether it can represent and apply relationships to configurations that were not directly observed during training.

2.2 First-Order Logic

First-order logic (FOL) provides the formal language underlying Logic Tensor Networks. It allows knowledge to be expressed in terms of individuals, properties, relations, functions, and quantifiers. A first-order language contains non-logical symbols such as constants, functions, and predicates. Constants refer to individuals in a domain, while predicates express properties or relations involving these individuals. For example,

$$P(x)$$

can represent a property of an individual x , whereas

$$R(x, y)$$

can represent a relation between two individuals x and y . First-order logic also provides logical connectives and quantifiers.

The universal quantifier

$$\forall x$$

expresses that a formula holds for every relevant individual, while the existential quantifier

$$\exists x$$

expresses that it holds for at least one individual.

These elements can be combined into more complex formulas. For instance,

$$\forall x (P(x) \rightarrow Q(x))$$

expresses that every individual satisfying P also satisfies Q .

The relevance of first-order logic to neurosymbolic learning lies in its ability to represent relationships explicitly. Rather than considering observations independently, a logical theory can describe properties and relations that should hold across multiple individuals. LTN uses this logical structure while replacing classical truth evaluation with a differentiable many-valued interpretation [1].

2.3 Fuzzy Logic and Differentiable Semantics

Classical logic assigns binary truth values to formulas. LTN instead employs a many-valued semantics in which formulas receive values in the continuous interval

$$[0, 1].$$

A value close to 1 represents a high degree of truth, whereas a value close to 0 represents a low degree of truth. Intermediate values allow formulas to be partially satisfied. This continuous representation makes it possible to associate logical operators with differentiable functions. Logical connectives can be implemented through fuzzy operators, while quantified formulas can be evaluated through aggregation mechanisms over their grounded instances [1]. For a formula φ , its interpretation can therefore be written as

$$\mathcal{G}(\varphi) \in [0, 1].$$

Because the resulting operations are differentiable, the truth value of a logical formula can depend on learnable parameters and can consequently be incorporated into gradient-based optimization.

The specific choice of fuzzy operators is relevant to the behaviour of differentiable logical systems. Different operators can have different mathematical and optimization properties, which can affect the resulting learning dynamics [3].

In the context of this thesis, this observation motivates keeping the logical semantics fixed across the experimental conditions whenever possible. The objective is to investigate the effect of structural representation rather than differences caused by alternative fuzzy operators.

2.4 Logic Tensor Networks

Logic Tensor Networks (LTN) provide a framework for combining first-order logic with differentiable tensor computation. The framework, developed from earlier work on learning and reasoning with Logic Tensor Networks [6], provides differentiable interpretations of logical symbols and formulas [1].

The central mechanism is *grounding*. Logical symbols are associated with numerical representations or differentiable functions. A constant representing an individual can, for example, be grounded into a vector, while a predicate can be grounded into a function that evaluates the degree to which its arguments satisfy the corresponding property or relation.

For a unary predicate P , its grounding can be represented abstractly as

$$G(P) : \mathbb{R}^d \rightarrow [0, 1].$$

For a binary relation R , the corresponding grounding can be written as

$$G(R) : \mathbb{R}^d \times \mathbb{R}^d \rightarrow [0, 1].$$

The output represents the degree to which the corresponding predicate or relation is satisfied. Once logical symbols have been grounded, complete formulas can be evaluated using the differentiable semantics introduced above. Consequently, a logical theory

$$\mathcal{T} = \{\varphi_1, \dots, \varphi_n\}$$

can be associated with degrees of satisfaction for its constituent formulas. Learning can then be formulated as the optimization of an objective that encourages the desired logical constraints to be satisfied.

Conceptually, this can be expressed as

$$\max_{\theta} \text{Sat}_{\theta}(\mathcal{T}),$$

where θ denotes the learnable parameters and $\text{Sat}_{\theta}(\mathcal{T})$ represents the degree of satisfaction of the theory under the current interpretation.

This formulation allows logical knowledge to influence learning rather than being used only as a post-training verification mechanism. Neural representations and logical constraints are therefore optimized within the same differentiable framework [1].

2.5 Structural Information in Learning

Many learning problems involve data whose organization contributes to its meaning. A temporal sequence is a simple example. Given

$$x_1 \rightarrow x_2 \rightarrow x_3,$$

the observations are not merely an unordered collection: their positions define relationships between them. The same distinction appears in other structured domains, such as graphs,

where connections between entities form part of the information relevant to the task.

This creates a representational issue for learning systems. If the target concept depends on the organization of the observations, representing the individual observations alone may not provide an explicit representation of the relevant relationships.

The original LTN framework provides a mechanism for representing logical relations between individuals, but does not explicitly introduce structural dimensions such as sequence positions as separate elements of the logical language. Structural Logic Tensor Networks (sLTN) extend the framework in this direction by making structural dimensions explicit [5].

2.6 Structural Logic Tensor Networks

Structural Logic Tensor Networks (sLTN) extend LTN by treating structural dimensions as first-class elements of the framework. A structural dimension can represent an axis associated with the organization of data, such as temporal steps or sequence positions [5].

The framework introduces structural variables and structural relations in addition to ordinary logical variables and relations. While ordinary variables range over individuals, structural variables range over positions within a structural dimension. Structural relations can then describe relationships between these positions.

For a temporal structure, for example, a relation such as

$$\text{next}(t, t')$$

can represent the fact that position t' immediately follows position t [5].

This distinction allows logical formulas to refer explicitly to the organization of the data. A formula can therefore express not only that an entity has a certain property, but also that the property occurs at a position related to another position according to the structure being represented.

sLTN preserves the differentiable semantics of LTN while extending the representation with structural information. Importantly, when structural dimensions are not used, the framework can recover the original LTN setting [5]. This makes the two frameworks naturally

comparable in an experimental setting.

The introduction of structural representations, however, does not imply that they must improve performance in every task. If a target can be determined without considering structural relationships, explicitly representing those relationships may provide little additional information. The potential benefit of sLTN should therefore depend on the role that structure plays in the task.

2.7 Systematic Generalization

A central concept for the experimental setting of this thesis is *systematic generalization*. In supervised learning, a model can achieve high performance on test examples that resemble its training examples without necessarily demonstrating that it can apply learned relationships to novel configurations.

Lake and Baroni studied this issue using the SCAN benchmark, which evaluates sequence-to-sequence models on combinations of commands that can differ from those observed during training. Their results demonstrated that good performance on familiar configurations does not necessarily imply systematic generalization to novel combinations [4].

The distinction is relevant to the present work because structural generalization can be evaluated by deliberately separating the structural configurations encountered during training from those used at test time.

For example, a model could be trained using sequences such as

$$A \rightarrow B \rightarrow C$$

and

$$A \rightarrow B \rightarrow D,$$

while being evaluated on configurations such as

$$A \rightarrow C \rightarrow B \rightarrow D.$$

The individual elements may already be familiar to the model, while their structural arrangement is new.

In this thesis, *structural generalization* is used as an operational term for this experimental condition: the ability of a model to generalize from structural configurations observed during training to previously unseen configurations at test time.

This definition is intentionally specific to the experimental setting of the thesis. It provides a measurable distinction between performance on familiar structural configurations and performance on novel ones, without assuming that structural generalization constitutes a separate established theoretical category in the literature.

2.8 Structural Dependency

The potential usefulness of explicit structural representations depends on whether the target task actually requires structural information.

Consider two sequence-based classification tasks. In the first, the target depends only on which elements occur in the sequence. The order of those elements does not affect the correct output. In the second, the target depends on the relative position or ordering of elements. In this case, information about the sequence structure is necessary to determine the target. This thesis refers to this distinction as *structural dependency*. Structural dependency describes the extent to which solving a task requires information about relationships involving positions, order, or other structural properties of the input.

A task with low structural dependency can be solved primarily from properties of individual observations. A task with high structural dependency requires information about how those observations are organized.

This distinction is important for evaluating sLTN because it provides a controlled way of asking when explicit structural representation should matter. If structure is irrelevant to the target, an explicit structural representation is not expected to provide substantial additional information. Conversely, when the target directly depends on structural relationships, representing those relationships explicitly may provide a more direct correspondence between

the model representation and the task.

Structural dependency is therefore treated as an experimental factor rather than as an assumption about the superiority of one model over another.

2.9 Structural Generalization and Data Availability

The amount of training data provides a second factor that may affect the ability of a model to learn structural relationships. With more training examples, a model may encounter a larger variety of structural configurations. With fewer examples, it has less opportunity to learn such configurations directly from the data.

Logical constraints provide an alternative source of information because they can express relationships that are not limited to individual training examples [1]. However, the presence of such constraints does not automatically guarantee improved data efficiency. Their usefulness depends on their relevance to the task and on how they interact with the learning objective.

For this reason, training-data availability is treated as a secondary factor in the experiments. The primary focus remains structural generalization, while different amounts of training data are used to investigate whether the effect of explicit structural representation changes when fewer examples are available.

The relevant notion of data efficiency in this thesis is therefore statistical: the amount of training data required to obtain a given level of generalization performance. It does not refer to computational efficiency or training time.

2.10 Research Gap

The theoretical background presented above establishes three relevant observations.

First, Logic Tensor Networks provide a differentiable framework in which first-order logical knowledge can be combined with neural representations and used directly during learning [1].

Second, sLTN extends this framework by making structural dimensions and structural re-

lations explicit, providing a representation for information such as sequence positions and temporal relationships [5].

Third, research on systematic generalization shows that successful learning on familiar configurations does not necessarily imply successful generalization to novel configurations [4].

Taken together, these observations motivate a more specific empirical question. The availability of an explicit structural representation does not by itself establish when such a representation produces a measurable advantage in generalization.

In particular, it remains relevant to investigate whether the effect of structural representation depends on the relationship between the target task and the underlying structure, on whether the structural configurations encountered at test time were observed during training, and on the amount of training data available.

This thesis addresses this problem through a controlled comparison between a neural baseline, conventional LTN, and sLTN. The experiments vary structural dependency and distinguish between structural configurations observed during training and previously unseen configurations at test time, while training-data availability is considered as a secondary factor.

The objective is therefore not to assume that sLTN is universally superior to conventional LTN or to neural models. Instead, the objective is to determine the conditions under which explicit structural representation contributes to generalization and to characterize situations in which its contribution is limited.

Chapter 3

Structural Logic Tensor Networks

Structural Logic Tensor Networks (sLTN) extend the Logic Tensor Network (LTN) framework by introducing an explicit representation of the structural organization of data. While the original LTN framework provides a differentiable logical language for reasoning over collections of individuals, many learning problems involve additional forms of organization, such as temporal order, sequence position, spatial structure, or graph connectivity. In these settings, the relationships between positions may constitute part of the information that a model needs to reason about. sLTN addresses this limitation by making structural organization an explicit component of the logical language [5]. The central idea of sLTN is to introduce structural dimensions as first-class elements of the language. Structural dimensions correspond to named axes of structured data, such as time steps in a temporal sequence, positions in a sequence, or nodes in a graph. Structural variables can range over these dimensions, while structural relations describe relationships between structural positions. Logical formulas can therefore refer not only to ordinary individuals, but also to the organization in which those individuals occur [5]. The framework preserves the main principles of LTN. Logical symbols are grounded into tensors or differentiable functions, while logical connectives and quantifiers are interpreted through differentiable fuzzy semantics. Structural constraints can therefore participate in gradient-based optimization. sLTN should consequently be understood as an extension of the LTN framework to structured representations rather than as a replacement for its underlying neurosymbolic approach [5].

3.1 Motivation and Limitations of Standard LTN for Structured Data

The original LTN framework is primarily designed for settings in which data can be represented as collections of individuals. Each individual belongs to a logical domain and can be associated with tensor representations, predicates, functions, and other logical symbols. This formulation is suitable when the relevant relationships can be expressed directly in terms of the individuals being represented. However, many datasets contain an additional organization that cannot be naturally described only in terms of independent individuals. Examples include temporal sequences, where observations are ordered in time; token sequences, where the position of an element is meaningful; and graphs, where connectivity between nodes contributes to the structure of the data. In such cases, structural information is not merely an implementation detail of the tensor representation, but may affect the meaning of the logical constraints themselves [5]. For example, consider a sequence of observations indexed by time. A property holding at time t may be related to the same property at a subsequent time t' . Expressing such a dependency requires the logical language to refer explicitly to the relationship between the two positions. Similarly, in a graph, a property associated with one node may need to be related to the properties of neighboring nodes. Such relationships can be represented indirectly by encoding structural information through ordinary individuals and predicates. However, this does not make the structural organization itself an explicit component of the logical language. sLTN instead introduces structural dimensions and structural relations so that these forms of organization can be represented directly within logical formulas [5]. This distinction is particularly relevant to the present thesis. By making structural organization explicit, sLTN provides a formal mechanism for investigating whether structural information can contribute to generalization when a model is evaluated on structural configurations that were not observed during training. The purpose of the framework is therefore not only to represent structured data, but also to make structural information available to differentiable logical reasoning.

3.2 Structural Dimensions

A structural dimension is a named dimension associated with a particular organization of data. Typical examples include a temporal dimension representing time steps, a sequence dimension representing positions in an ordered sequence, or a graph dimension representing nodes. Structural dimensions provide an explicit representation of the axes along which structured data are organized [5]. In sLTN, structural dimensions are part of the formal signature of the language. The signature extends the standard components of a many-sorted first-order language with structural dimensions and structural variables. The dimensional profile associated with a symbol specifies which structural dimensions are relevant to its tensorial representation [5]. This establishes a connection between the logical representation and the underlying tensor representation. A tensor may contain one or more axes corresponding to structural dimensions, and these axes are explicitly named and tracked by the framework. Structural information can consequently be used when expressions are aligned, selected, masked, or aggregated. For example, suppose that a video is represented as a sequence of frames. A temporal dimension T can be associated with the sequence of frames. A tensor representing a batch of videos may therefore contain an axis corresponding to T . Rather than treating this axis as an anonymous tensor dimension, sLTN associates it with a named structural dimension that can be referenced by logical expressions. The explicit naming of dimensions is important because the numerical shape of an axis does not determine its semantic meaning. An axis representing time is conceptually different from an axis representing image width, even if both are represented numerically as tensor dimensions. sLTN makes this structural meaning explicit within the language and its semantics [5].

3.3 Structural Variables

Structural variables are used to refer to individual positions of a structural dimension and are distinct from ordinary first-order variables [5]. An ordinary first-order variable ranges over individuals belonging to a declared logical sort. For example, a variable x of sort *Image* may range over images in a dataset. A structural variable, in contrast, ranges over positions

associated with a structural dimension. If t is associated with a temporal dimension, it can represent a particular position in time. This distinction allows a formula to refer simultaneously to an individual and to the structural position at which that individual is observed. For example, a property can be associated with an object x at time t , while another property can be associated with the same object at a different time t' . Structural variables can also be quantified. The language can therefore express statements over all or selected positions of a structural dimension. This is particularly useful when a logical constraint depends on relationships between multiple structural positions rather than on a single observation [5]. The distinction between first-order and structural variables is consequently fundamental to sLTN: first-order variables describe the entities being reasoned about, whereas structural variables describe where those entities occur within an explicitly represented structure.

3.4 Structural Relations

Structural dimensions identify positions, but structured reasoning also requires a mechanism for expressing relationships between those positions. sLTN introduces structural relations for this purpose. A structural relation connects elements of one or more structural dimensions. Such relations may represent temporal adjacency, sequence ordering, or graph connectivity. They are interpreted as Boolean or fuzzy masks over structural dimensions and can therefore be incorporated into the differentiable semantics of the framework [5]. A simple example is the temporal relation $next(t, t')$, which identifies pairs of consecutive positions in a temporal dimension. This relation can be used to express a constraint such as

$$\forall t, t' \mid next(t, t') : (A(x, t) \rightarrow A(x, t')) .$$

The formula states that whenever the relation $next(t, t')$ holds, the property A should be preserved from one temporal position to the next. The relevant aspect is that temporal adjacency is represented directly by a structural relation and can consequently be referenced by the logical constraint [5]. Structural relations therefore provide the mechanism through which dependencies between positions become accessible to logical reasoning. The same

principle can be applied to other structures. In a graph, for example, a relation can identify neighboring nodes, while in a sequence it can represent consecutive positions or another predefined ordering. Because structural relations may be represented by Boolean or fuzzy masks, they can be incorporated into the differentiable semantics of sLTN. Structural knowledge can therefore be combined with learned representations without requiring the structural component to be treated as an entirely separate symbolic preprocessing stage [5].

3.5 Syntax and Semantics

The formal language of sLTN extends a many-sorted first-order signature with structural components. The signature contains sorts, dimensions, constants, individual variables, functions, predicates, structural variables, and structural relations. Typing information additionally specifies the structural dimensions associated with symbols and the dimensions consumed or produced by functions and predicates [5]. A central aspect of the framework is the separation between the syntactic description of a theory and its tensorial interpretation. Symbols are first declared in a signature. Formulas are then constructed and checked against this signature before concrete tensor representations are assigned through an interpretation [5]. This separation allows the logical structure of a theory to be specified independently of the particular tensors used during learning. It also makes structural information subject to explicit consistency checks at the language level. Function and predicate applications must be compatible with their declared sorts and structural profiles, while structural annotations and quantifiers must refer to appropriate structural variables and dimensions. The concrete syntax of sLTN supports logical operators and quantifiers in both symbolic and textual forms. In addition to ordinary first-order quantification, the language supports structural quantification, allowing formulas to quantify explicitly over positions belonging to structural dimensions [5]. The resulting framework can therefore express formulas in which ordinary logical entities and structural positions coexist. A formula can describe a property of an individual, constrain that property across different structural positions, or condition a statement on a structural relation between those positions.

3.6 Tensor Representation and Named Axes

A central characteristic of sLTN is the direct connection between structural information and tensor dimensions. The framework interprets terms and formulas as annotated tensors whose axes may correspond to ordinary variables, structural dimensions, and domain dimensions [5]. Consider a collection of videos represented by a tensor. If a batch contains 32 videos, each consisting of 8 frames, the corresponding tensor may contain a batch axis, a temporal structural axis, and image-domain axes. The temporal axis can be explicitly associated with a structural dimension such as T . Named structural axes allow expressions to be aligned according to their semantic dimensions rather than relying only on their numerical tensor positions. This becomes relevant when formulas combine expressions carrying different structural profiles. Structural operations can therefore include alignment, broadcasting, selection, masking, and aggregation along explicitly identified axes. These operations provide a connection between the logical representation of structure and its tensorial implementation [5]. The design also preserves compatibility with ordinary LTN representations. When no structural dimensions are associated with an expression, the additional structural machinery is absent and the corresponding interpretation reduces to the unstructured case. More generally, sLTN is designed so that the original LTN semantics is recovered as a special case when structural dimensions are not used [5].

3.7 Logical Reasoning over Structural Information

The introduction of structural dimensions and structural relations extends the types of constraints that can be expressed within the logical language. Standard LTN allows logical formulas to describe relationships between individuals and constrain their learned representations. sLTN additionally allows such formulas to refer explicitly to the organization of the underlying data [5]. This makes it possible to express constraints involving temporal persistence, sequence dependencies, or relationships between connected structural positions. A structural relation acts as a logical condition identifying which combinations of positions should be considered related. For example, let $A(x, t)$ denote a property of an individual x

at temporal position t . A persistence constraint can be expressed using a relation $next(t, t')$:

$$\forall t, t' \mid next(t, t') : (A(x, t) \rightarrow A(x, t')) .$$

The structural relation determines where the logical implication is applied. Consequently, the formula does not simply require A to hold independently at all positions. Instead, it specifies a relationship between the truth of A at two structurally connected positions [5]. The same mechanism can represent more general structural dependencies. For instance, if a property depends on the order of elements in a sequence, the logical theory can refer to the corresponding structural positions and their relationships. In a graph setting, a relation describing adjacency can determine which nodes participate in a particular logical constraint. Structural information can therefore be combined with learned representations within the same differentiable logical framework. Logical constraints remain subject to fuzzy semantics and gradient-based optimization, while their applicability can depend explicitly on the organization of the structured input [5].

3.8 Temporal Structures as a Case Study

Temporal data provide a natural example of the motivation behind sLTN because temporal order is an intrinsic part of the representation. The sLTN paper uses a video classification example in which a temporal dimension represents the sequence of frames in a video [5]. In this setting, a video can be associated with a structural dimension T , whose positions correspond to individual frames. A frame can therefore be represented not only by its visual content but also by its position along the temporal dimension. A structural relation such as $next(t, t')$ can identify consecutive frames. Logical formulas can then use this relation to impose constraints across adjacent positions. For example, if a property is expected to persist through consecutive frames, this dependency can be expressed explicitly using the temporal relation [5]. The temporal example illustrates the main conceptual difference introduced by sLTN. Temporal organization is not treated solely as part of the underlying tensor or neural architecture; it becomes accessible to the logical language and can therefore participate

directly in differentiable logical constraints. This case is particularly relevant to the research question of this thesis because sequences provide a controlled setting in which structural organization can be manipulated independently of the underlying individual elements. Such a setting makes it possible to investigate whether explicitly representing structural information affects the ability of a neurosymbolic model to generalize to structural configurations that were not observed during training.

3.9 Relationship between LTN and sLTN

sLTN should be understood as an extension of LTN rather than as a separate alternative framework. Both approaches rely on the same general neurosymbolic principle: logical expressions are grounded into differentiable tensorial representations, while logical constraints are evaluated using fuzzy semantics so that they can participate in gradient-based optimization [1, 5]. The main difference concerns the representation of structure. Standard LTN provides a logical language for reasoning over individuals and their relationships, whereas sLTN introduces structural dimensions, structural variables, and structural relations as explicit components of the language [5]. This distinction can also be understood in terms of tensor representation. In standard LTN, tensor dimensions primarily support the grounding of logical entities. In sLTN, selected dimensions can additionally be assigned an explicit structural meaning, which can then be referenced by the logical language. An important consequence is that the original LTN setting remains available as a special case. When no structural dimensions are used, the additional structural machinery is absent and the corresponding unstructured LTN semantics is recovered [5]. This relationship is relevant to the experimental comparison developed in the following chapter. Since sLTN extends the same general neurosymbolic framework while making structural information explicit, the comparison between LTN and sLTN can be used to investigate whether explicit structural representation produces measurable differences when the learning problem depends on structural information.

3.10 Limitations and Open Issues of sLTN

Although sLTN introduces a formal mechanism for representing structural information, the framework is presented as an initial framework and software library rather than as a fully validated solution for structured neurosymbolic learning. The authors explicitly identify the need for further stabilization, testing, and systematic empirical validation beyond the illustrative examples presented in the paper [5]. Several directions are identified as future work. One concerns the development of dedicated structured theories for concrete application domains, in which domain-specific objects, predicates, dimensions, and structural relations can be defined systematically. Another concerns extensions of the language with finitely recursive computational primitives while preserving differentiability and a well-defined tensor semantics [5]. The paper also identifies the need for principled strategies for selecting or scheduling the power-mean exponent used in aggregation, with the objective of improving training stability and supporting more systematic hyperparameter selection. Further work is proposed on benchmarking the library components and comparing sLTN with alternative approaches for relational, temporal, spatial, and graph-based learning [5]. These open issues are particularly relevant to the present thesis. The introduction of explicit structural representations provides a formal mechanism for reasoning about structured data, but the empirical conditions under which this representation is actually beneficial remain an open question. In particular, the existence of a structural language does not by itself establish that explicit structural reasoning improves generalization or reduces the amount of data required for learning. The following chapter therefore moves from the formal framework to an empirical investigation. Rather than assuming that structural representations are always advantageous, the experimental methodology is designed to examine whether their effect depends on the degree to which the target task relies on structural information and on the availability of training data.

Chapter 4

Experimental Methodology

This chapter describes the experimental methodology used to investigate whether and under which conditions explicit structural representations improve the generalization of neurosymbolic learning systems. The experimental design is motivated by the research questions and hypotheses introduced in Chapter 1 and by the distinction between conventional Logic Tensor Networks (LTN) and Structural Logic Tensor Networks (sLTN) discussed in Chapters 2 and 3.

The central objective is not to assume that sLTN is universally superior to conventional LTN or to neural models. Instead, the experiments are designed to isolate the effect of explicit structural representation by controlling the task, the data distribution, the degree of structural dependency, the novelty of the structural configurations encountered at test time, and the amount of training data.

The experimental study therefore focuses on three model settings: a conventional neural baseline, an LTN model, and an sLTN model. All three models are evaluated on the same controlled sequence classification problem. The experiments vary the degree to which the target label depends on structural relationships within the sequence and distinguish between structural configurations observed during training and previously unseen configurations at test time.

4.1 Experimental Objective and Research Questions

The main research objective is to determine whether explicitly representing structural information within a neurosymbolic model leads to measurable improvements in structural

generalization.

More specifically, the experiments address the following questions:

RQ1 — Structural Generalization. To what extent does explicit structural reasoning affect the ability of a model to generalize from observed structural configurations to previously unseen configurations?

RQ2 — Model Comparison. How does the performance of a Structural Logic Tensor Network compare with that of a conventional Logic Tensor Network and a neural baseline when generalizing to previously unseen structural configurations?

RQ3 — Conditions for Structural Benefit. How does the effect of explicit structural representation vary with the degree of structural dependency of the target task and with the amount of training data available?

These questions are investigated using a controlled synthetic task. The use of a controlled setting is intentional: it allows structural dependency and structural novelty to be manipulated independently of domain-specific complexity. Consequently, differences between the models can be interpreted in relation to the structural properties of the task rather than to unrelated characteristics of a real-world dataset.

4.2 Experimental Task

The experimental task is a binary classification problem over fixed-length symbolic sequences. Each example is represented as a sequence of length eight:

$$x = (x_1, x_2, \dots, x_8),$$

where each element belongs to a finite alphabet

$$\mathcal{A} = \{A, B, C, D, E, F, G, H\}.$$

The symbols A , B , and C are used to define the class-dependent structural patterns, while the remaining symbols are used as filler elements.

Each sequence is associated with a binary target label

$$y \in \{0, 1\}.$$

The important property of the task is that the same basic observations can participate in different classification rules depending on the structural condition being evaluated. In particular, the experiments distinguish between tasks in which the order of elements is irrelevant, tasks in which relative order matters, and tasks in which a specific local structural configuration is required.

The sequence length and alphabet are kept fixed across all experimental conditions. Only the rule defining the target label and the corresponding structural dependency are varied.

4.3 Structural Dependency

The main independent variable of the experimental design is the degree of structural dependency of the classification task. Three levels are considered: Low, Medium, and High.

This factor is central to the research questions because the potential benefit of an explicit structural representation should depend on whether the target variable actually requires structural information.

4.3.1 Low Structural Dependency

In the Low condition, the target depends on the presence of two relevant symbols but not on their relative positions.

A sequence is assigned the positive class when it contains both A and C :

$$y_{\text{low}}(x) = \mathbb{I}[A \in x \wedge C \in x].$$

The relative order of A and C is irrelevant. For example, both

$$(A, D, E, C, F, G, H, B)$$

and

$$(C, D, A, E, F, G, H, B)$$

satisfy the positive condition.

The Low condition therefore provides a control setting in which the input contains structural information, but the target classification does not require reasoning about that structure.

The corresponding implementation used to generate the labels is:

```
def low_label(seq):
    return int("A" in seq and "C" in seq)
```

This condition is important for distinguishing the benefit of structural representation from the mere presence of additional model machinery. If explicit structural reasoning is useful only when the target depends on structural relationships, little or no advantage is expected in this condition.

4.3.2 Medium Structural Dependency

In the Medium condition, the target depends on the relative ordering of two elements. A sequence is positive when A occurs before C :

$$y_{\text{medium}}(x) = \mathbb{I}[\text{pos}(A) < \text{pos}(C)].$$

The absolute positions of the two symbols are not themselves the target. Instead, the relevant information is the structural relation between them.

The corresponding implementation is:

```
def medium_label(seq):
    return int(seq.index("A") < seq.index("C"))
```

The Medium condition therefore represents an intermediate level of structural dependency. Solving the task requires information that is absent from an unordered representation of the elements.

In the sLTN formulation, the relevant structural relation can be represented by a relation such as

$$before(t, t'),$$

which holds when structural position t precedes structural position t' .

The corresponding logical pattern can conceptually be expressed as

$$\exists t, t' (before(t, t') \wedge A(x[t]) \wedge C(x[t'])).$$

4.3.3 High Structural Dependency

The High condition requires a more specific local structural configuration. A sequence is positive when it contains the consecutive pattern

$$A \rightarrow B \rightarrow C.$$

Formally,

$$y_{\text{high}}(x) = \mathbb{I}[\exists t : (x_t, x_{t+1}, x_{t+2}) = (A, B, C)].$$

The corresponding label-generation function is:

```
def high_label(seq):
    return int(
        any(
            seq[i:i+3] == ("A", "B", "C")
            for i in range(len(seq) - 2)
        )
    )
```

)

The High condition introduces an explicit local structural dependency. The relevant information is not only the presence of the three symbols or their general order, but their occurrence in three consecutive structural positions.

In the sLTN formulation, this relationship is represented using the structural relation

$$\text{next}(t, t'),$$

together with a logical constraint connecting three consecutive positions. Conceptually, the corresponding pattern can be expressed as

$$\exists t, t', t'' (\text{next}(t, t') \wedge \text{next}(t', t'') \wedge A(x[t]) \wedge B(x[t']) \wedge C(x[t''])).$$

The High condition is consequently intended to represent the strongest form of structural dependency considered in the experiment.

4.4 Dataset Construction

The dataset is generated synthetically in order to provide precise control over the structural properties of the examples. The generator is deterministic given a random seed and explicitly separates training examples from the structural configurations reserved for evaluation.

Each dependency condition contains three subsets:

1. a training set;
2. a seen test set;
3. an unseen test set.

The training and seen test sets are generated from structural configurations that are available during training, whereas the unseen test set contains structural configurations that are not observed during training.

Each generated subset contains an equal number of positive and negative examples. In the full-data condition, the generated training set contains 500 examples per class, resulting in 1000 training examples. The seen and unseen evaluation sets contain 500 examples each, resulting in balanced evaluation sets of the same size.

The resulting basic dataset configuration is therefore:

Table 4.1: Dataset sizes used in the experiments.

Subset	Positive	Negative	Total
Training	500	500	1000
Seen test	250	250	500
Unseen test	250	250	500

The exact sequence-generation procedure additionally enforces disjointness between the training, seen, and unseen sets. This prevents the same complete sequence from appearing in multiple subsets.

4.5 Structural Signatures

A central aspect of the dataset generator is the use of structural signatures. A structural signature represents the structural configuration relevant to the corresponding dependency condition.

For the Low and Medium conditions, the signature is based on the positions of the relevant symbols. Missing symbols are represented using a reserved value. For the High condition, the signature identifies the structural location of the controlled local pattern.

The structural signature is not itself the target label. Instead, it is used to control which configurations are available during training and which are reserved for testing.

This distinction is important because the experiment is intended to evaluate generalization across structural configurations rather than memorization of individual sequences.

4.6 Structural Train–Test Split

The main generalization manipulation distinguishes between structural configurations observed during training and configurations that are structurally novel at test time.

The split is performed at the level of structural configurations rather than by independently sampling complete sequences. This ensures that an unseen example can contain familiar symbols and satisfy the same general classification rule while differing in the structural configuration through which that rule must be applied.

For the Low and Medium conditions, structural signatures are partitioned into training and held-out groups. Examples belonging to the held-out signatures are assigned to the unseen test set.

For the High condition, the controlled structural pattern is associated with specific positions within the sequence. Some structural positions are used during training, while other positions are reserved for the unseen evaluation condition.

The resulting distinction can be summarized as follows:

Seen structural configurations: configurations represented during training and reused for evaluation;

Unseen structural configurations: configurations not represented during training but evaluated at test time.

This protocol is designed to measure structural generalization rather than ordinary interpolation between independently sampled examples.

4.7 Example Disjointness and Leakage Control

A potential source of experimental leakage would be the presence of identical sequences in more than one dataset partition. The generator therefore explicitly checks exact-sequence disjointness.

For subsets S_i and S_j , the required condition is

$$S_i \cap S_j = \emptyset \quad \forall i \neq j.$$

In addition, structural leakage is checked by verifying that configurations assigned to the unseen partition are not present in the training partition.

These checks are performed automatically before model training. The purpose is to ensure that unseen accuracy measures generalization to structurally held-out configurations rather than memorization of examples appearing in the training set.

4.8 Model Configurations

Three model settings are compared:

1. a neural baseline;
2. a conventional Logic Tensor Network;
3. a Structural Logic Tensor Network.

The three settings are evaluated on the same generated data and under the same training-data conditions.

4.8.1 Neural Baseline

The neural baseline provides a reference model without explicit logical reasoning. The sequence is represented numerically and processed by a neural classification architecture producing a scalar probability for the positive class.

The purpose of this baseline is to determine whether the performance of LTN or sLTN differs from a conventional data-driven neural approach.

The neural model does not explicitly represent the relations *before* or *next* at the logical level.

4.8.2 Logic Tensor Network

The second model is a conventional LTN. It uses the differentiable logical framework described in Chapter 2 but does not introduce explicit structural dimensions and structural relations as part of the logical representation.

The LTN model therefore provides the most direct comparison with sLTN. Both models belong to the same general neurosymbolic framework, while sLTN additionally exposes structural information to the logical language.

This comparison is particularly important because a difference between a neural baseline and sLTN alone would not establish that the explicit structural representation is responsible for the effect. The LTN condition provides an intermediate reference point.

4.8.3 Structural Logic Tensor Network

The third model uses the sLTN framework described in Chapter 3. Structural dimensions and structural variables are explicitly associated with the sequence positions.

The sequence is therefore represented using a structural dimension T , while structural variables range over positions in this dimension. Structural relations such as *before* and *next* are used to express the relevant dependencies.

For example, the logical structure used for the Medium condition is based on the relation

$$before(t, t')$$

and connects the corresponding symbol predicates at the two structural positions.

For the High condition, the logical structure connects three positions through the adjacency relation *next*.

The sLTN objective combines the supervised classification objective with the satisfaction of the structural logical constraints. In simplified form, the training objective can be written as

$$\mathcal{L} = \mathcal{L}_{\text{sup}} + \lambda \mathcal{L}_{\text{struct}},$$

where $\mathcal{L} * \text{sup}$ is the binary classification loss, $\mathcal{L} * \text{struct}$ measures the violation of the structural logical constraints, and λ controls the contribution of the structural component.

The corresponding training principle implemented in the experimental notebook is represented by the following simplified code:

```
pred = model(x)

supervised = binary_cross_entropy(pred, y)
structural = model.structural_loss(x)

loss = supervised + STRUCTURAL_WEIGHT * structural
```

The structural loss is therefore part of optimization rather than being used only for post-hoc evaluation. This distinction is essential for testing whether structural information actually affects the learning process.

4.9 Logical Constraints

The logical constraints are adapted to the structural dependency of each task.

For the Low condition, the relevant rule does not require a relationship between two positions. The condition is that an A occurs somewhere in the sequence and a C occurs somewhere in the sequence:

$$(\exists t A(x[t])) \wedge (\exists t' C(x[t'])).$$

For the Medium condition, the structural relation between the two positions becomes relevant:

$$\exists t, t' (\text{before}(t, t') \wedge A(x[t]) \wedge C(x[t'])).$$

For the High condition, the rule explicitly refers to consecutive positions:

$$\exists t, t', t'' (next(t, t') \wedge next(t', t'') \wedge A(x[t]) \wedge B(x[t']) \wedge C(x[t''])).$$

These constraints are intended to provide the structural information that differentiates sLTN from the neural baseline and conventional LTN.

4.10 Training Data Availability

Structural generalization is the primary focus of the study. Training-data availability is introduced as a secondary experimental factor to investigate whether the effect of structural representation changes when fewer examples are available.

Three training-data conditions are considered:

- 100
- 50
- 25

The test sets are kept unchanged across these conditions. Only the amount of training data available to the model is reduced.

This design allows the experiments to distinguish between two questions. First, whether a model can generalize to structurally unseen configurations. Second, whether the amount of data required to achieve such generalization changes when explicit structural information is available.

The notion of data efficiency used here is therefore statistical rather than computational. It refers to predictive performance under different quantities of training data and does not refer to training time or hardware efficiency.

4.11 Experimental Matrix

The complete experimental design combines three structural dependency conditions, three training-data conditions, and three model settings.

Table 4.2: Experimental conditions.

Structural dependency	100%	50%	25%
Low	✓	✓	✓
Medium	✓	✓	✓
High	✓	✓	✓

Each configuration is evaluated using five independent random seeds. Consequently, the complete experiment contains

$$3 \times 3 \times 3 \times 5 = 135$$

individual training runs.

The experiment can therefore be viewed as a controlled factorial design with:

- model type as one factor;
- structural dependency as one factor;
- training-data availability as one factor;
- random initialization as the repeated stochastic factor.

4.12 Random Seeds and Reproducibility

Neural and neurosymbolic models are stochastic systems. Their results can vary depending on parameter initialization and other sources of randomness. To reduce the influence of a single initialization, each experimental condition is repeated using five random seeds.

The seeds are fixed across model comparisons. This produces paired experimental conditions in which, for example, the Neural, LTN, and sLTN models evaluated under the same dependency and data condition are trained using corresponding random seeds.

This paired structure is useful for comparing models because it reduces the influence of differences caused solely by random initialization.

The experimental notebook records the model, structural dependency, training-data fraction, seed, and evaluation results for every run.

4.13 Evaluation Metrics

The primary evaluation metrics are accuracy on seen structural configurations, accuracy on unseen structural configurations, and the corresponding generalization gap.

4.13.1 Seen Accuracy

Seen accuracy measures classification performance on structural configurations that were available during training:

$$Accuracy_{\text{seen}} = \frac{1}{N_{\text{seen}}} \sum_{i=1}^{N_{\text{seen}}} \mathbb{I}[\hat{y}_i = y_i].$$

This metric measures ordinary predictive performance under structurally familiar conditions.

4.13.2 Unseen Accuracy

Unseen accuracy measures performance on structural configurations that were held out during training:

$$Accuracy_{\text{unseen}} = \frac{1}{N_{\text{unseen}}} \sum_{i=1}^{N_{\text{unseen}}} \mathbb{I}[\hat{y}_i = y_i].$$

This is the primary metric for the structural generalization analysis.

4.13.3 Generalization Gap

To quantify the degradation associated with structural novelty, the following generalization gap is used:

$$GeneralizationGap = Accuracy_{\text{seen}} - Accuracy_{\text{unseen}}.$$

A small gap indicates that performance is relatively stable when moving from familiar to unseen structural configurations. A large gap indicates that performance decreases substantially under structural novelty.

The metric is particularly useful for the High condition, where a model may achieve high performance on familiar structural configurations while failing to transfer the learned relation to unseen configurations.

4.14 Data Efficiency Analysis

The effect of training-data availability is analyzed by comparing performance across the 100

The analysis does not assume that a reduction in training data will necessarily benefit a particular model. Instead, it examines whether the relative differences between Neural, LTN, and sLTN change as the amount of available training data decreases.

In particular, the analysis considers whether a model maintains unseen structural accuracy when fewer training examples are available.

The data-efficiency analysis therefore complements the main structural-generalization analysis rather than replacing it.

4.15 Statistical Analysis

For each combination of model, dependency, and training-data condition, results from the five random seeds are aggregated using the mean and standard deviation.

For a metric m measured across $K = 5$ seeds, the mean is computed as

$$\bar{m} = \frac{1}{K} \sum_{k=1}^K m_k,$$

while the standard deviation is used to describe the variability across runs.

The primary comparisons concern:

1. Neural versus LTN;
2. Neural versus sLTN;
3. LTN versus sLTN;
4. the change in these differences across structural dependency levels;

5. the change in these differences across training-data conditions.

The main outcome of interest is unseen accuracy, while the generalization gap provides a complementary measure of structural transfer.

Because only five seeds are used for each condition, statistical tests are interpreted cautiously. In particular, a difference in mean performance is not automatically interpreted as evidence of a robust model advantage. Variability across seeds is reported alongside the mean values.

4.16 Experimental Hypotheses

The experimental design is based on the hypotheses introduced in Chapter 1.

4.16.1 H1 — Structural Generalization

Explicit structural representations are expected to improve generalization when the target task depends on structural relationships.

This hypothesis is evaluated primarily through unseen accuracy and generalization gap.

4.16.2 H2 — Dependence on Structural Dependency

The relative effect of explicit structural representation is expected to increase as the degree of structural dependency increases.

The comparison between Low, Medium, and High conditions is therefore central to this hypothesis.

4.16.3 H3 — Training Data Availability

The potential benefit of explicit structural representation is expected to become more relevant when fewer training examples are available.

This hypothesis is investigated through the 100

4.16.4 H4 — Structural Relevance

When structural information is not required to solve the target task, explicit structural reasoning is not expected to provide a systematic advantage.

The Low condition provides the main control for this hypothesis.

4.17 Implementation and Experimental Reproducibility

The complete experimental pipeline is implemented in a single executable notebook. The notebook performs dataset generation, integrity checks, model initialization, training, evaluation, result aggregation, and visualization.

The experimental configuration is centralized so that the main parameters can be reproduced consistently. For example, the sequence length and alphabet are defined explicitly:

```
SYMBOLS = tuple("ABCDEFGH")
SEQ_LEN = 8
DEFAULT_SEED = 42
N_PER_CLASS = 500
```

The complete dataset generator also performs automatic checks for class balance, exact-example disjointness, and structural leakage before training begins.

The notebook records the results of every individual run in a structured table. This makes it possible to reconstruct aggregate results from the individual seed-level measurements rather than relying only on precomputed averages.

The use of a single notebook also reduces the risk of inconsistencies between separately executed scripts during the experimental phase. The complete implementation is retained as part of the experimental artifacts, while selected code fragments are included in this thesis to document the methodology.

4.18 Experimental Controls

Several controls are incorporated into the experimental design to isolate the effect of explicit structural representation.

First, all models are evaluated on the same generated examples. Differences in model performance therefore cannot be attributed to differences in the data presented to the models.

Second, the same structural train–test partition is used across model comparisons. The unseen condition consequently represents the same structural generalization challenge for Neural, LTN, and sLTN.

Third, the training-data fractions are applied consistently across models. A 25

Fourth, multiple random seeds are used for every experimental configuration.

Finally, the distinction between LTN and sLTN is kept conceptually explicit. LTN represents the conventional logical framework without the additional structural representation, while sLTN introduces structural dimensions and structural relations into the logical language.

These controls are necessary because the purpose of the experiment is not simply to compare three architectures, but to investigate whether explicit structural reasoning contributes to generalization under controlled conditions.

4.19 Experimental Interpretation Protocol

The results are interpreted according to the experimental design rather than according to an assumption that one model must outperform the others.

In particular, an improvement of sLTN over the neural baseline is not by itself sufficient to establish the usefulness of structural reasoning. The effect must also be considered in relation to the LTN baseline and to the structural dependency of the task.

Similarly, a failure of sLTN to outperform the other models is not interpreted as evidence that structural reasoning is ineffective in general. It may instead indicate that the particular structural representation, logical formulation, or task configuration does not provide a useful inductive bias under the considered conditions.

The experimental analysis therefore focuses on conditional effects. The central question is whether the difference between models changes systematically as the relevance of structural information changes.

This approach is consistent with the overall research objective of the thesis: rather than determining whether sLTN is universally superior, the experiments investigate the conditions under which explicit structural representation contributes to generalization and the conditions under which its contribution is limited.

Chapter 5

Experimental Results

What happened in the experiments — tables, plots, accuracy/F1, generalization results

Chapter 6

Analysis of Structural Generalization

Interpret the results specifically in terms of when structural reasoning helps

Chapter 7

Discussion

Answer RQs/Hypotheses, compare findings with LTN/sLTN literature, explain implications and limitations

Chapter 8

Conclusions and future research

Final findings, contribution, limitations, future research

Bibliography

Articles and papers

- [1] Samy Badreddine et al. “Logic Tensor Networks”. In: *Artificial Intelligence* 303 (2022), p. 103649. ISSN: 0004-3702. DOI: [10.1016/j.artint.2021.103649](https://doi.org/10.1016/j.artint.2021.103649). URL: <https://doi.org/10.1016/j.artint.2021.103649> (cit. on pp. 1–3, 7, 13–16, 20, 29).
- [2] Artur d’Avila Garcez and Luís C. Lamb. “Neurosymbolic AI: The 3rd Wave”. In: *Artificial Intelligence Review* 56.11 (2023), pp. 12387–12406. DOI: [10.1007/s10462-023-10448-w](https://doi.org/10.1007/s10462-023-10448-w). URL: <https://doi.org/10.1007/s10462-023-10448-w> (cit. on p. 13).
- [3] Emile van Krieken, Erman Acar, and Frank van Harmelen. “Analyzing Differentiable Fuzzy Logic Operators”. In: *Artificial Intelligence* 302 (2022), p. 103602. DOI: [10.1016/j.artint.2021.103602](https://doi.org/10.1016/j.artint.2021.103602). URL: <https://doi.org/10.1016/j.artint.2021.103602> (cit. on p. 15).
- [4] Brenden M. Lake and Marco Baroni. “Generalization Without Systematicity: On the Compositional Skills of Sequence-to-Sequence Recurrent Networks”. In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. PMLR, 2018, pp. 2873–2882. URL: <https://proceedings.mlr.press/v80/lake18a.html> (cit. on pp. 12, 18, 21).
- [5] Davide Rinaldi and Luciano Serafini. *sLTN: Structural Logic Tensor Networks*. 2026. arXiv: [2608.11136](https://arxiv.org/abs/2608.11136). URL: <https://arxiv.org/abs/2608.11136> (cit. on pp. 2, 4–7, 17, 21–30).
- [6] Luciano Serafini and Artur d’Avila Garcez. “Learning and Reasoning with Logic Tensor Networks”. In: *AI*IA 2016 Advances in Artificial Intelligence*. Springer, 2016, pp. 334–348. DOI: [10.1007/978-3-319-49130-1_25](https://doi.org/10.1007/978-3-319-49130-1_25). URL: https://doi.org/10.1007/978-3-319-49130-1_25 (cit. on p. 15).